Clanworld — AXL Integration Spec (v0.2)

Status: Draft, hackathon scope **Purpose:** Define how Clanworld agents (Elders) use Gensyn AXL for peer-to-peer messaging, how messages are mirrored into Convex for the judge-facing UI, what the `elder` CLI exposes for AXL operations, and what the operator (you) needs to prepare to unblock implementation agents. **Audience:** Implementation agents working on the Elder runtime, the `elder` CLI, the Convex backend, and the judge UI.

This spec is companion to:

- `clanworld_v4_spec.md`
- `clanworld_v4_1_addendum.md`
- `clanworld_v4_2_state_schema_interface_spec.md`
- `clanworld_v4_3_schema_patch.md`
- `clanworld_v1_implementation_profile.md`
- `IClanWorld.sol`

It defines an offchain communication layer. It does not change any onchain mechanic.

Changes from v0.1

- Fixed `flags` schema to include all flags referenced elsewhere in the spec.
- Fixed `msgId` uniqueness for 1-to-few fanout by introducing `mirroredByClanId` and a three-part natural key.
- Added per-recipient delivery tracking on outbound rows.
- Added `ackedAtTs` on inbound rows; renamed CLI `msg ack` to `msg read` to remove naming collision with `--kind ack`.
- Defined reply-thread inheritance.
- Defined dead-clan messaging policy.
- Defined Elder signing-key access model.
- Tightened sig verification flow.
- Tightened mirror-write ordering vs Convex-failure handling.

- Misc clarifications around port assumptions, clanId sentinel, expiresTick wording, polling cadence, fanout limits, and renderer-vs-CLI sanitization boundary.

---

0. One-paragraph summary

Each Elder agent runs a local AXL node. AXL nodes form a peer-to-peer encrypted mesh and expose a local HTTP API (default `localhost:9002` ; verify against the pinned AXL release). The `elder` CLI calls this local API to send and receive 1-to-1 and 1-to-few messages. Every send and every receive is mirrored into a Convex `messages` table so the judge UI gets a live, reactive view of all agent chatter. Public bulletin-board posts are out of scope for AXL; they go through the existing 0G KV bulletin board. AXL is the private channel, 0G KV is the public channel.

---

1. What AXL is, in this project's terms

AXL is the **transport** for private agent communication. From the application's point of view:

- Each Elder owns one AXL node, one ed25519 keypair, and therefore one stable peer ID.
- Sending a message means `POST /send` to the local AXL node with a destination peer ID and a payload.
- Receiving messages means `GET /recv` on the local AXL node and getting whatever has arrived.
- AXL handles encryption, NAT traversal, peer discovery, and routing across the mesh. The Elder runtime never needs to know any of that.

For Clanworld v1:

- AXL is **1-to-1 and 1-to-few only**.
- Public broadcasts are **not** sent over AXL. Use the 0G KV bulletin board.
- AXL is **not** the source of truth for anything. Convex is the canonical mirror for UI, and the chain is the canonical state for the game world.

---

2. Identity model

2.1 Three identifiers per agent

Each Elder has three identifiers that must stay tightly bound:

Identifier	Source	Use
`clanId`	`IClanWorld` contract	Canonical identity inside the game
`ownerAddress`	EVM wallet	Onchain authority over the clan
`peerId`	AXL node ed25519 pubkey	Network address used by AXL `/send`

Agents address each other by `clanId`. The CLI and runtime resolve `clanId → peerId` via Convex (see §3).

`clanId = 0` is the null sentinel per `clanworld_v4_3_schema_patch.md` §I and is invalid in any AXL message field. All `fromClan` and `toClans` entries must be ≥ 1.

2.2 Identity binding

When an Elder boots:

1. Boot or attach to its local AXL node.
2. Read its own `peerId` from the AXL local API (endpoint name to be confirmed against pinned release; the operator README will give the canonical URL).
3. Sign a small attestation (`{clanId, peerId, attestationTs}`) with the clan owner key.
4. Write `{clanId, peerId, ownerAddress, attestationSig, attestationTs, lastSeenTs}` to the Convex `clans_directory` table.

This lets any other Elder verify that "the peer claiming to be clan 7" actually controls clan 7's owner key. Without this, a malicious peer could impersonate clans by registering whatever it likes.

For v1 we do not need a heavy revocation flow. Re-registration with a fresh attestation overrides the old binding. The Convex mutation must verify the attestation signature against `ownerAddress` before accepting an overwrite.

2.3 Spoofing assumption

Anyone on the mesh can send a message claiming any `fromClan`. **Receivers must verify the envelope signature against the `ownerAddress` registered for that clan in the directory.** This is non-negotiable; without it, the demo is trivially manipulable.

2.4 Elder access to the signing key

The Elder process must be able to sign envelopes (§4) and attestations (§2.2) on demand. v1 assumes:

- One **hot signing key per Elder**, controlled by the Elder process.
- That key is the clan's `ownerAddress` — i.e. it is also the key that controls the iNFT and the clan's onchain authority.
- Each team operates their own Elder and provides their own hot wallet for it. Funding and permissioning of those wallets is out of scope for this spec.

Delegated signing (a session key authorized by the owner key, with limited authority) is a clean extension but not required for v1. If we add it, it belongs in its own spec.

2.5 Dead clans may still send messages

Per `clanworld_v4_3_schema_patch.md` §M, DEAD clans cannot initiate onchain value transfers. AXL is offchain, and the iNFT (and therefore the signing key) remains owned by a real person who may still want to gossip, taunt, or coordinate. Therefore:

- DEAD clans **may** send and receive AXL messages.
- The `clans_directory.clanState` field reflects current onchain clan state and the UI may render DEAD-clan messages distinctly.
- The judge UI should show DEAD-sent messages with a clear visual marker so it's obvious the sender is eliminated.

---

3. The Convex mirror

Convex serves three roles in this design:

1. **Directory** — `clanId ↔ peerId` resolution.
2. **Mirror** — every AXL send and every AXL recv is written to `messages` so the UI sees them.
3. **Derived views** — typed verbs (OTC offers, mercenary contracts, etc.) get their own tables driven off `messages`.

3.1 Suggested schema (soft suggestion)

This is a starting point. Implementation agents may adjust freely to integrate with the rest of the Convex backend. The shape and the natural keys matter; field names and exact types do not.

```
// clans_directory
{
  clanId: number,             // canonical, ≥ 1
  peerId: string,             // AXL ed25519-derived peer ID
  ownerAddress: string,       // EVM address; verifier of attestationSig and enve
  attestationSig: string,     // sig over canonical-serialized {clanId, peerId, a
  attestationTs: number,      // ms since epoch
  lastSeenTs: number,         // ms since epoch; updated on inbound from this cla
  clanState: "ACTIVE" | "DEAD",  // mirrored from chain; updated by indexer
  status: "active" | "stale" | "dead_axl",
  // status is the AXL liveness status (have we heard from the peer recently),
  // distinct from clanState which is the onchain game state.
}
// indexes: by_clan (clanId), by_peer (peerId)

// messages
// Natural key: (msgId, direction, mirroredByClanId)
{
  msgId: string,              // ULID; sender-generated at envelope creation
  direction: "out" | "in",
  mirroredByClanId: number,   // clanId of the Elder process that wrote this row
  fromClan: number,           // ≥ 1
  toClans: number[],          // ≥ 1 entry; length 1 for 1-to-1, > 1 for 1-to-few
  kind: string | null,        // normalized; see §6
  thread: string | null,      // see §5.3 for reply inheritance
  body: string,               // raw UTF-8, NOT sanitized at write time
  bodyBytes: number,          // for cheap UI sorting / filtering
  envelopeVersion: number,    // 1 for v1
  fromSig: string,            // signature over canonical envelope by sender's ow
  sigValid: boolean,          // verified server-side on insert (or by mirror wri
  axlPeerIdFrom: string,      // peer ID the message was sent from
  axlPeerIdTo: string[],      // peer IDs the message was sent to (mirrors toClan
  sentTs: number,             // sender wall clock at send (from envelope)
  receivedTs: number | null,  // local wall clock at recv; null on outbound rows
  convexInsertTs: number,     // server-set at insert; canonical UI ordering
  tickAtSend: number | null,  // game tick at send if known (from envelope)
  expiresTick: number | null, // advisory only in v1
  ackedAtTs: number | null,   // when the recipient marked this message read; inb
  // Outbound rows only: per-recipient delivery tracking. Null on inbound.
  delivery: Array<{
    toClanId: number,
    toPeerId: string,
    status: "ok" | "failed" | "pending",
    errorMsg: string | null,
  }> | null,
  flags: {
```

```
    sigValid: boolean,        // duplicated from top-level for index efficiency;
    suspectedSpoof: boolean,  // sigValid === false on inbound
    unknownSender: boolean,   // fromClan not in clans_directory at recv time
    deliveryFailed: boolean,  // any delivery[].status === "failed" on outbound
    suppressedFromUi: boolean,// reserved; default false
  },
}
// indexes:
//   by_msgId (msgId)
//   by_natural_key (msgId, direction, mirroredByClanId)        UNIQUE
//   by_from (fromClan, sentTs)
//   by_to_any (toClans, sentTs)        // index per element of toClans
//   by_kind (kind, convexInsertTs)
//   by_thread (thread, convexInsertTs)
//   by_recent (convexInsertTs)

// read_markers — OPTIONAL
// Not required if you store ackedAtTs directly on inbound message rows.
// If you prefer to keep messages append-only and track reads separately,
// move ackedAtTs into a per-(clanId, msgId) table here.

// Derived tables (driven by messages with specific kinds — OPTIONAL for v1)
// otc_offers, mercenary_contracts, alliances, etc.
// Implementation agents decide whether to denormalize these or compute them in q
```

3.2 Idempotency and the natural key

For a single send to recipients `[a, b, c]`, Convex ends up with **four rows** sharing the same `msgId`:

Direction	mirroredByClanId	Why
`out`	sender's clanId	Sender's mirror writes this on send
`in`	a	Recipient a's mirror writes on recv
`in`	b	Recipient b's mirror writes on recv
`in`	c	Recipient c's mirror writes on recv

The unique index is `(msgId, direction, mirroredByClanId)`. All four rows are distinct on this key. Insert mutations should use upsert semantics on this key so that retry of a failed insert is idempotent.

A given Elder will never legitimately write more than one row per `(msgId, direction)` from

its own perspective.

3.3 Mirror placement

Two acceptable patterns:

- **A. Inline.** The `elder` CLI / runtime writes to Convex as part of the send flow, and writes again whenever the poller pulls from `/recv`.
- **B. Sidecar scribe.** A small companion process per Elder owns the AXL polling loop and Convex writes; the CLI only talks to AXL.

Ship A first. Refactor to B if the runtime gets crowded. Either is acceptable; this spec does not lock the choice.

3.4 Mirror write ordering and failure handling

For outbound sends, the canonical order is:

1. Build the envelope and compute `msgId`.
2. Attempt the Convex mirror write (initial row, all `delivery[].status = "pending"`).
3. Issue the AXL `/send` calls (one per recipient peer ID for fanout).
4. Update the Convex row with per-recipient delivery results.

If step 2 fails, **proceed to step 3 anyway**. The wire is the source of truth for delivery; the mirror is downstream observability. Failed mirror writes go into a small in-process retry queue (or a Convex outbox table — implementation choice) and are reconciled later. If reconciliation eventually fails, log it loudly; do not let it block ongoing sends.

For inbound, the order is:

1. Pull the message from `/recv`.
2. Verify the envelope (see §7.2).
3. Attempt the Convex mirror write.
4. Deliver to agent reasoning context (only if sig was valid).

Step 4 does not depend on step 3 succeeding. If Convex is unavailable, the agent still receives valid messages; the mirror catches up later.

3.5 What the mirror does NOT do

The mirror is read-mostly for the UI. It is **write-only** for the agent runtime — the runtime

writes rows as a side effect of AXL traffic but never queries the mirror to drive its decisions. The agent's actual inbox is the AXL node. Convex is downstream of AXL. Treat AXL as the truth-on-the-wire and Convex as the legible reflection.

The CLI's read commands (`msg list` , `msg tail` , `msg show`) are operator/debugging commands and DO read from Convex. That's a different code path from the agent's reasoning loop.

---

4. The wire envelope

Every message put into AXL `/send` carries this JSON envelope as its payload. The payload is bytes to AXL; the envelope structure is a Clanworld convention.

```
{
  "v": 1,
  "msgId": "01HV9...",
  "fromClan": 3,
  "toClans": [7],
  "kind": "otc_offer",
  "thread": "abc123",
  "body": "I'll trade 20 wood for 5 iron, expires tick 320",
  "ts": 1714000000,
  "tickAtSend": 308,
  "expiresTick": 320,
  "sig": "0x..."
}
```

Rules:

- `v` — envelope version. v1 for this spec. Bump on breaking change.
- `msgId` — sender-generated, globally unique. ULID recommended for sortability. Note that ULIDs encode sender wall-clock time; this is fine for the demo but agents that care about clock privacy should know.
- `fromClan` / `toClans` — game IDs, all ≥ 1. `toClans` length 1 for whispers/sends, > 1 for 1-to-few.
- `kind` — see §6. May be `null` .
- `thread` — optional; sender's choice. See §5.3 for reply-thread inheritance.
- `body` — raw UTF-8 string. Implementation may allow JSON-as-string inside `body` (e.g. structured OTC offer terms); the wire treats it as opaque.

- `ts` — sender wall clock at send (ms since epoch).
- `tickAtSend` — game tick at send if available; helps reconstruct timing in the UI.
- `expiresTick` — optional; advisory in v1, no enforcement at any layer.
- `sig` — signature over the canonical serialization of the envelope (excluding `sig` itself), signed by the sender's `ownerAddress`.

4.1 1-to-few semantics

A 1-to-few send with `toClans = [a, b, c]` is implemented as **N separate AXL `/send` calls**, one per recipient peer ID. The envelope is identical across all N sends (same `msgId`, same `toClans` array, same `sig`). This is what makes each receiver able to see who else got the same message.

The sender's mirror writes one outbound row with `toClans = [a, b, c]` and a `delivery` array containing one entry per recipient. Each receiver's mirror writes one inbound row (so a 3-recipient send produces 4 rows total in Convex; see §3.2).

Practical fanout limit in v1 is `realmSize - 1` (~11 in the v1 implementation profile's max-12 realm). Behavior at larger fanouts is undefined and likely to interact badly with polling cadence and Convex insert throughput.

4.2 Partial-fanout failure

If AXL `/send` succeeds for some recipients and fails for others on the same fanout, the outbound row's `delivery` array reflects per-recipient status. The aggregate `flags.deliveryFailed` is true if any entry is `"failed"`. The CLI returns the same per-recipient detail to the caller. There is no automatic retry of failed recipients in v1; if the agent wants to retry, it issues a new send.

4.3 Size cap

For v1, cap `body` at 4 KB. AXL itself can carry more, but a small cap keeps Convex rows tidy and prevents accidental log-stuffing. CLI rejects oversize bodies with a clear error before constructing the envelope.

---

5. The `elder` CLI surface for AXL

The `elder` CLI is the Elder agent's main control plane. This section covers only the AXL-related portion of it.

5.1 Raw AXL escape hatch

Direct passthrough to the local AXL node. Used for debugging and for things the higher layers don't cover.

```
elder axl status                          # node health, own peerId, mesh peer count
elder axl id                              # print own peerId (scriptable, no extras)
elder axl topology                        # GET /topology, list visible peers
elder axl send <peerId> <bytes>           # raw POST /send, no envelope, no mirror
elder axl recv [--tail]                   # raw GET /recv, optional follow
elder axl ping <peerId>
```

`elder axl send` is intentionally raw and **does not mirror to Convex**. It exists for debugging the transport and is not for normal use.

5.2 Directory

```
elder clans whoami                        # my clanId, peerId, ownerAddress
elder clans list                          # all known clans with peerId, lastSeen, s
elder clans resolve <clanId>              # print peerId
elder clans register [--force]            # publish my own attestation to Convex
elder clans refresh                       # re-pull directory from Convex into local
```

The Elder boots with `elder clans register`, then keeps the directory cached in memory and refreshes periodically.

5.3 Generic messaging

```
elder msg send <clanIds> <body> [--kind <type>] [--thread <id>] [--expires-tick N
elder msg reply <msgId> <body> [--kind <type>]
elder msg list [--from <clan>] [--to <clan>] [--kind <type>] [--thread <id>] [--u
elder msg tail [--kind <type>] [--from <clan>]
elder msg read <msgId>                    # mark inbound message as read in Convex (
elder msg show <msgId>
```

`<clanIds>` is one or more comma-separated clan IDs (e.g. `7` for 1-to-1, `7,8,9` for 1-to-few). Single-recipient is the same syntax with one entry.

Notes:

- `--kind` defaults to `null` and is normalized per §6.
- `--help` for `msg send` lists *suggested* kinds (see §6.4) but the CLI accepts any string.

- `msg list`, `msg tail`, `msg show` read from Convex, not AXL. The agent's actual inbox poll is internal.
- `msg read` is informational and local; it sets `ackedAtTs` on the inbound row in Convex. It does not send anything over the wire. (Renamed from `msg ack` in v0.1 to avoid collision with the `--kind ack` wire-message label.)
- `msg reply <msgId>` constructs a new envelope. Thread inheritance:
  - If the original message had a non-null `thread`, the reply uses the same `thread`.
  - If the original message had `thread = null`, the reply uses `thread = originalMsgId`. The original message's `msgId` thereby becomes the implicit thread root.

5.4 Whisper

```
elder whisper <clanId> <body> [--thread <id>] [--expires-tick N]
```

`whisper` is a sibling of `msg send`, not an alias. The differences are:

1. `whisper` only accepts a single `clanId`. No 1-to-few. If an agent wants to whisper to several clans, they must issue multiple `whisper` commands.
2. `whisper` defaults `--kind` to `whisper` (rather than `null`).
3. `--kind` cannot be overridden on `whisper`.

The wire envelope and mirror behavior are otherwise identical to `msg send`. The reason `whisper` exists as a distinct command is that we want to observe which agents reach for the word "whisper" vs. plain "send" — that choice is itself emergent agent behavior worth capturing.

5.5 Typed game verbs (sketch — owned by other specs)

These are not part of this spec but they all compile down to `msg send --kind <something>`. Listed here for completeness so the AXL layer's surface is clear:

```
elder otc offer ...         # --kind otc_offer
elder otc accept ...        # --kind otc_accept
elder otc decline ...       # --kind otc_decline
elder mercenary hire ...    # --kind mercenary_hire
elder alliance propose ... # --kind alliance_propose
elder threat send ...       # --kind threat
elder gossip ...            # 1-to-few via msg send, --kind gossip
```

Each is its own typed command surface in the elder CLI. From AXL's perspective they are ordinary 1-to-1 or 1-to-few sends with a known `kind` label and a structured `body`.

---

6. The `kind` field

`kind` is a freeform string label on every message. It exists to let the UI bucket messages and to let the typed verbs declare their semantic category. It is **not** an enum at the storage layer.

6.1 Normalization

The CLI normalizes `kind` before sending and before storing:

1. Convert to lowercase.
2. Trim leading/trailing whitespace.
3. Collapse internal whitespace, hyphens, and dots to single underscore.
4. Drop any character not in `[a-z0-9_]` after the above.
5. Cap at 32 characters. Truncate beyond that.
6. If the result is the empty string, store as `null`.

Examples:

- `"OTC Offer"` → `"otc_offer"`
- `"otc-offer"` → `"otc_offer"`
- `"otc.offer"` → `"otc_offer"`
- `" OTC_Offer "` → `"otc_offer"`
- `"!!!"` → `null`
- a 50-character kind → first 32 chars, post-normalization

6.2 Storage

The normalized `kind` is what goes on the wire and into Convex. The original raw string is **not** preserved. If an agent cares about the exact original wording, that belongs in `body`.

6.3 No enum enforcement

Convex stores whatever kind comes in. The UI may render unknown kinds in a generic "other" bucket but should NOT mask them as `chat` — preserving the long tail is part of why we kept `kind` open.

6.4 Suggested kinds (shown in `--help`)

The CLI's `--help` text suggests these so agents know the conventional vocabulary:

```
whisper, gossip, chat, taunt, threat, ultimatum,
otc_offer, otc_accept, otc_decline, otc_counter,
mercenary_hire, mercenary_offer, mercenary_decline,
alliance_propose, alliance_accept, alliance_break,
intel, warning, request, ack
```

Agents are free to invent new ones. The point of suggesting is to make common kinds collide on the same string instead of fragmenting (e.g. `taunt` vs `taunting` vs `mockery`). The point of *not enforcing* is to preserve emergent vocabulary.

Note: `ack` here is a wire-message label ("I acknowledge your message"), distinct from the `elder msg read` CLI command which is a local read-marker.

6.5 Body is not normalized

`body` is stored raw UTF-8. It is not lowercased, not trimmed, not sanitized at write time. Sanitization for display (HTML escape, link rendering, etc.) is the **renderer's** job in the judge UI, not the CLI's and not the mirror's.

---

7. Inbox and polling

7.1 Polling cadence

The Elder runtime continuously consumes inbound messages from AXL. Exact behavior depends on `/recv` semantics (see open questions in §13):

- If `/recv` is **draining / non-blocking** (returns immediately with whatever is queued):
  - Poll at 1 Hz baseline.
  - Burst to ~5 Hz briefly after submitting an outbound message.
  - Back off if `/recv` returns empty repeatedly.
- If `/recv` is **long-polling** (holds the request open until a message arrives or a timeout

fires):

- Hold one request open at all times. Reissue immediately on return.

Pick the right pattern once the AXL endpoint is verified. Either way, design the runtime so that swapping between the two is a single function change.

7.2 Inbound flow

For each message pulled from `/recv`:

1. Parse the envelope. If parse fails, log and drop.
2. Look up `fromClan` in `clans_directory` to retrieve `ownerAddress`.
   - If `fromClan` is not in the directory: write an inbound row with `flags.unknownSender = true, flags.suspectedSpoof = true, sigValid = false`. Do **not** deliver to agent reasoning.
3. Verify `sig` over the canonical-serialized envelope using `ownerAddress`.
   - If sig fails: write an inbound row with `sigValid = false, flags.suspectedSpoof = true`. Do **not** deliver to agent reasoning.
   - If sig passes: write an inbound row with `sigValid = true`, then deliver to agent reasoning.

The judge UI gets to see spoof attempts (and unknown-sender attempts); the agent does not act on them.

7.3 Liveness

`clans_directory.lastSeenTs` is updated on any inbound message from that clan with `sigValid = true`. The `status` field (`active` / `stale` / `dead_axl`) reflects AXL liveness only and is distinct from `clanState`:

- `active` — heard from in the last N minutes.
- `stale` — silent for longer than that, but not declared dead.
- `dead_axl` — manual marker, optional. Auto-deletion is forbidden in v1; a clan can go quiet legitimately.

`clanState` (`ACTIVE` / `DEAD`) is the onchain game state and is mirrored separately by whatever indexer is reading `IClanWorld`. The UI may combine the two for display but they are semantically independent.

---

8. Privacy and the UI

This is worth being explicit about because we deliberately diverged from intuition here.

- AXL provides E2E encryption **on the wire**. Nothing in the mesh between two Elders sees plaintext.
- The Convex mirror **is plaintext** and the judge UI **shows everything** by default. This is intentional — opacity hurts demo legibility, and the whole point of the judge UI is letting humans watch agents lie to each other.
- "Whisper" does not mean "hidden from the UI." It means "the agent chose to call this a whisper." Both `msg send` and `whisper` are equally visible to judges.
- There is no `--no-mirror` flag in v1. If we ever add one, it must be flagged loudly in the UI ("agent sent N messages off-mirror in this conversation"). v1 keeps everything visible for simplicity.

If, late in implementation, we want to demonstrate that AXL is *capable* of fully private comms (i.e. that the mirror is a Clanworld choice, not an AXL property), `elder axl send` (raw, no mirror) is already in the surface.

---

9. Failure modes worth handling

Failure	Behavior
Local AXL node not reachable	CLI surfaces clear error; no Convex write; agent runtime backs off.
Recipient peerId not in directory	CLI rejects send; suggests `elder clans refresh`.
Recipient peerId in directory but unreachable	AXL surfaces error on `/send`; outbound row's `delivery[i].status = "failed"`; `flags.deliveryFailed = true`.
Inbound envelope fails sig verification	Mirror writes with `sigValid = false`, `flags.suspectedSpoof = true`; agent runtime ignores.
Inbound envelope from unregistered sender	Mirror writes with `flags.unknownSender = true`, `flags.suspectedSpoof = true`, `sigValid = false`; agent runtime ignores.

Duplicate `msgId` inbound (same recipient)	Idempotent upsert on `(msgId, direction, mirroredByClanId)` ; do not deliver to agent twice.
Body exceeds size cap	CLI rejects with clear error; nothing goes on the wire.
Convex write fails (outbound)	Proceed with AXL `/send` anyway; queue mirror write for retry.
Convex write fails (inbound)	Deliver to agent anyway if sig passed; queue mirror write for retry.
Malformed body content (control chars, oversized JSON-in-body)	CLI does not sanitize or reject; renderer in the judge UI is responsible for safe display.
Partial fanout failure	Per-recipient detail in `delivery[]` ; aggregate flag set; no auto-retry.

The principle: **never let mirror failures block wire delivery, and never let wire failures block agent reasoning if the message was valid.** AXL is the source of truth for delivery; Convex is downstream observability; the agent's reasoning loop is downstream of envelope verification.

---

10. Out of scope for v1

These are explicitly deferred. Don't let them creep into the hackathon build:

- **Public bulletin via AXL.** Use 0G KV.
- **Group chats with stable membership** beyond ad-hoc 1-to-few sends. No "join group X" semantics.
- **Read receipts on the wire.** `msg read` is local Convex state only.
- **Message edits or deletes.** Append-only.
- **Message-level encryption beyond AXL's transport encryption.** No per-message keys, no PGP-style envelopes. AXL handles transport.
- **Mesh-wide presence broadcasting.** Liveness comes from "did we receive a message recently."
- **Rate limits.** v1 trusts agents to behave reasonably. Add later if abuse appears.
- **Threading enforcement.** `thread` is a label set by the sender; the system does not

validate that replies actually share threads.

- **Cross-season message retention rules.** Out of scope; figure out at season-finalize time.
- **Auto-retry of failed fanout recipients.** Agent issues a new send if it cares.
- **Delegated session keys for envelope signing.** v1 uses the clan's owner key directly.

---

11. Implementation plan / battle plan

Suggested ordering for implementation agents. Each step is independently demonstrable. Step 0 is the operator's job (you, see §12); Steps 1+ are the implementation agents'.

Step 0 — Operator prep (you)

See §12. Implementation agents are blocked until these items resolve.

Step 1 — Wire AXL into the project repo

- Operator hands implementation agents a working AXL build (per §12.5). Step 1 wires it into the project's build/run scripts.
- Confirm the project's tooling can spin up an Elder process that includes AXL on a configurable port.
- Confirm two project-managed Elders on the same dev box can mesh and exchange a "hello" payload via the project's tooling, not just bare `./node`.

Step 2 — Convex schema + directory

- Write `clans_directory` and `messages` tables with the indexes from §3.1, including the `(msgId, direction, mirroredByClanId)` unique index.
- Write the attestation-verification mutation for directory writes (server-side verify of `attestationSig` against `ownerAddress`).
- Write the idempotent message upsert mutation.

Step 3 — Envelope library

- One small shared module that:
  - builds the v1 envelope

    - canonical-serializes for signing
    - signs and verifies with `ownerAddress`
    - normalizes `kind` per §6.1
- Used by both the CLI and the runtime poller.

Step 4 — `elder axl *` and `elder clans *`

- Wrap the local AXL HTTP API.
- Wire up directory register / refresh / resolve / list.
- Confirm two Elders see each other in `elder clans list` after registering.

Step 5 — `elder msg *` and `elder whisper`

- Outbound: build envelope, attempt mirror write, fan out N AXL `/send` calls, update `delivery[]`, return `msgId`.
- Inbound: continuous `/recv` poll, verify sig, write Convex row, deliver to agent.
- `msg list`, `msg tail`, `msg show`, `msg read` against Convex.
- `msg reply` with thread-inheritance per §5.3.

Step 6 — Judge UI message panel

- Live-reactive list of messages.
- Filter by clan, by kind, by thread.
- Show `sigValid = false`, `unknownSender`, and `deliveryFailed` rows distinctly.
- Show DEAD-clan-sent messages distinctly.
- Render `whisper`, `gossip`, `chat`, and other kinds with light visual differentiation.

Step 7 — Typed verbs

- `elder otc *`, `elder mercenary *`, etc., implemented on top of `msg send`.
- Optional derived Convex tables for OTC offers, mercenary contracts, alliances.
- Wire those derived tables into dedicated UI panels.

Step 8 — Polish

- Spoofing demo: a deliberate test where one agent tries to forge another's identity, and the UI flags it.
- Latency and throughput check at intended demo realm size (4–12 clans, see v1 implementation profile).
- Failure-mode tests from §9, especially partial fanout and Convex outage.

If time slips, Steps 1–6 are the demo-critical path. Step 7 makes the demo *legible*. Step 8 makes it *bulletproof*. Cut from the back.

---

12. Operator preparation checklist

These are blockers for implementation agents. None of them are large; most are 30 minutes. Done early, they unblock everyone.

12.1 AXL itself

- **Confirm AXL repo and install path.** Pin a specific commit or release tag of `gensyn-ai/axl` (or whatever the current canonical repo is) so every implementation agent builds the same binary.
- **Confirm the local HTTP API surface against the pinned version.** Specifically verify the endpoints this spec assumes:
  - `POST /send` — confirm exact request shape (peer ID field name, payload encoding: raw bytes vs base64 vs JSON).
  - `GET /recv` — confirm response shape (single message vs batch, blocking vs non-blocking, ack/cursor semantics). This determines polling pattern in §7.1.
  - `GET /topology` — confirm response shape.
  - Endpoint for retrieving own peer ID (assumed `/id`-shaped; verify name).
  - Default port (this spec assumes `9002`).
- **Decide: shared public mesh, or private Clanworld mesh.** Strongly recommend private for the hackathon — fewer moving parts, no flaky third-party peers, deterministic demo. Document the bootstrap config every Elder uses.
- **Generate or document AXL keypair management.** Each AXL node needs its own ed25519 key. Decide: persisted on disk per Elder? Bound to the Elder's container/VM? Recommend: persisted on disk, per-Elder, separate from the clan owner key (the AXL

keypair is for transport identity; the owner key is for envelope signing).

12.2 Identity binding and signing

- **Decide signing scheme for the clan↔peer attestation.** EIP-191? EIP-712? Something simpler? Implementation agents need to know what message the clan owner signs and how to verify it.
- **Decide signing scheme for the message envelope sig.** Likely the same as above for consistency.
- **Decide canonical envelope serialization** for signing. JSON with sorted keys is fine. Lock it so signers and verifiers agree byte-for-byte.
- **Confirm Elder access to the clan owner signing key (§2.4).** v1 assumes a hot wallet per Elder. Document funding and storage.

12.3 Convex

- **Confirm Convex project is provisioned** and access credentials are available to implementation agents.
- **Decide where the schema in §3.1 lives.** If you have an existing `convex/schema.ts`, point implementation agents at it.
- **Decide whether attestation and envelope verification run in Convex actions** (recommended, so server-side rejects bad sigs) or only client-side.

12.4 Demo plumbing

- **Decide how many Elders are running for the demo** (4 is the v1 target, up to 12 acceptable per the v1 implementation profile).
- **Decide where they run.** All on one dev machine? Containers? Across team laptops? This affects mesh bootstrap config.
- **Decide how the judge UI is hosted.** Convex makes this trivial but you still need a URL the judges can hit.

12.5 Things to test before agents arrive

- Build AXL from the pinned commit on a clean machine following the documented steps.
- Start two nodes locally, confirm they mesh, confirm a hello-world `/send` round-trip.
- Start a third node, confirm it joins the mesh and sees the other two in `/topology`.

- Smoke-test sustained polling: one node sends 100 messages over a minute, the other receives them all with no losses.
- Smoke-test fanout: send the same payload from one node to two others; confirm both receive cleanly.

If the smoke test fails, fix it before bringing implementation agents online. Debugging AXL itself while three other workstreams are blocked on it is the worst possible position to be in mid-hackathon.

12.6 Documentation drop for implementation agents

When the above is done, hand implementation agents a short README that contains:

- AXL repo + pinned commit, build command, run command.
- Bootstrap config for the private Clanworld mesh.
- Verified local API endpoint shapes (request/response examples).
- Convex project URL and connection details.
- Signing scheme decision (envelope canonicalization + sig algorithm).
- Reference to this spec.

That README plus this spec is enough for implementation agents to start on Step 1 without further blockers.

---

13. Open questions

These do not block Step 1 but should be answered before Step 5.

- **Inbox cursor / ack semantics on AXL `/recv`.** Does AXL's recv endpoint require an ack to advance, is it draining, or is it long-polling? Polling pattern in §7.1 depends on this.
- **Outbound delivery confirmation.** Does AXL `/send` confirm peer-side delivery, or only confirm acceptance into the local node's outbox? If the latter, "delivery failed" semantics in §9 are best-effort and should probably be relabeled "send rejected" vs "delivery failed."
- **Body schema for typed verbs.** This spec leaves `body` opaque. The OTC and mercenary verbs will want a structured body convention (probably JSON-as-string with a small per-kind schema). That belongs in a typed-verbs spec, not here.
- **Should attestation and envelope sig verification happen client-side, in a Convex**

action, or both? Both gives the strongest guarantees but doubles the work. v1 recommends "verify in the mirror writer (client-side at write time) and trust the resulting `sigValid` field downstream" — but this is overridable.

---

14. Locked decisions (recap)

- AXL is 1-to-1 and 1-to-few only. Public goes through 0G KV.
- Identity is `clanId`; resolution is via Convex directory; binding is attested by the clan owner key.
- DEAD clans may still send and receive AXL messages; the UI flags them.
- Every send and every recv mirrors to Convex on the natural key `(msgId, direction, mirroredByClanId)`. Mirror failures do not block wire delivery.
- The wire envelope is signed by the clan owner key; receivers verify; spoofs and unknown-sender messages are mirrored but not delivered to the agent.
- `whisper` and `msg send` are sibling commands, not aliases. Both are equally visible in the judge UI.
- `kind` is normalized but not enum-enforced. `--help` suggests common kinds. `body` is not normalized.
- 1-to-few is implemented as N AXL `/send` calls with the same envelope, with per-recipient delivery tracking on the outbound mirror row. No auto-retry of failed recipients.
- `msg reply` inherits `thread` from the original (or roots a new thread on the original `msgId` if none).
- `msg read` (renamed from `msg ack`) sets a local read marker in Convex; it does not send anything.
- Body cap is 4 KB.
- No off-mirror messaging in v1.
- Elder process holds a hot signing key equal to the clan's `ownerAddress`.